AXE 2.2

Library Reference

Copyright © 2011-2026 GB Research, LLC.

Contents

Introduction

Changes in v.2.2

Changes in v.2

Syntax Rules

- AXE operator rules
- AXE Shortcuts
- AXE operator""
- AXE functions and class constructors
- AXE Iterator transformation rules
- AXE Predicate Functions
- AXE Predicate Operators
- AXE polymorphic rule
- Limiting recursion depth
- Validating UTF-8
- AXE Parse Tree bindings
- AXE Utility functions
- AXE exceptions
- Creating custom predicates
- Creating custom rules
- Recursive rules

Semantic Actions

- AXE in-place extractors
- Creating custom semantic actions
- Using parse tree

Examples

- Zip code parser
- Telephone number parser
- CSV parser
- INI File parser
- Command line parser
- Windows Path Parser
- Roman Numerals
- JSON Parser
- Replacement Parser
- A word counting program
- Wildcard parser

Sources

Copyright © 2011-2026 GB Research, LLC.

Introduction

AXE is a C++ library that provides facilities to build recursive descent parsers. A recursive descent parser is a top-down parser built from a set of mutually-recursive procedures (or classes) where each such procedure (or class) implements one of the production rules of the grammar.[1]

AXE library contains a set of classes and functions to define syntax rules and semantic actions. The library uses standard C++ and doesn't require compiler/platform specific facilities. It needs C++17; C++20 is needed for `char8_t` input and for the compile-time string literals (see AXE operator""). It is tested with Visual Studio 2026 (`/std:c++latest`), GCC 14 and Clang 20 (`-std=c++23`), and each header also compiles on its own with GCC and Clang at C++17, C++20 and C++23. The README explains how to build and run the tests.

AXE is a header only library, it doesn't require linking. You only need to add

```
#include <axe.h>
```

in your source files and set the include directory in your compiler environment to point to `axe/include`.

---

1. Recursive descent parser: Wikipedia

Copyright © 2011-2026 GB Research, LLC.

Changes in v.2.2

Change	Explanation
String literals	With C++20 `"abc"_axe` is the compile-time rule `r_strlit<'a','b','c'>` on every compiler, for `char`, `wchar_t`, `char8_t`, `char16_t` and `char32_t` literals, and `""_axe` is `r_empty`. With C++17 it is `r_str`. Character literals `'c'_axe` create `r_char`. See AXE operator"".
Recursion depth	`r_depth_limit` bounds the nesting depth of recursive rules, so deeply nested input can't overflow the stack. See Limiting recursion depth.
UTF-8	`r_utf8` and `r_utf8str` match well-formed UTF-8. See Validating UTF-8.
Const rules	`&`, `-` and `|` report an operand without a const `operator()` with a `static_assert`.
`A && B`	Is now the same as `A & B | B & A`: B followed by A is also tried when A matches, but B doesn't follow it.
`r_bool`	A lambda without captures is called, not converted to `true`. A value or a function without arguments is evaluated also at the end of input.
Repetitions	`*A`, `+A`, `A % B` and `r_many` stop when an occurrence matches without consuming input, instead of repeating forever.
Parse trees	`r_named(A)` and `A/C` produce the parse tree of A for any rule A. `parse_tree` of `A % B` no longer consumes a trailing separator. `write_xml` closes `<optional>` correctly.
Binary input	`r_bin` and `r_lit` of a number match bytes of 0x80 or more also in `char` input; `r_var`, `r_array` and `r_sequence` accept `std::byte` input.
Iterator transformations	`r_skip` with a skipper rule that matches empty input, such as `*_ws`, no longer skips forever. `r_convert` accepts lambdas with captures also around rules that assign iterators, such as `+A`. `r_buffered` compiles without warnings at `-Wextra`.
Extraction	Extraction to `std::string_view` works with any contiguous iterator in C++20. `get_as` of an empty range returns a value-initialized object.
Tests	The tests build with Visual Studio and, with the Makefile, with GCC and Clang, and cover the rules that differed from this reference and the iterator transformation rules.

Copyright © 2011-2026 GB Research, LLC.

Changes in v.2

AXE v.2 introduced some significant changes compared to v.1

Version 2	Explanation
compiler	AXE v.2 requires reasonably conforming C++17 compiler. AXE v.1 required minimum C++11 compiler.
Composite rules	In AXE v.2 all composite rules take sub-rules by value. Which means that l-value rules are copied, and r-value rules are moved. In AXE v.1 the l-value sub-rules were held by reference. That created a problem of dangling references when returning a rule from a function. AXE v.2 parsers no longer have that limitation and can be returned from the functions. User can still hold sub-rules by reference using `std::reference_wrapper` by explicitly calling `std::ref`. **This change can break the v.1 code.** As a result of this change the recursive use of `axe::r_rule` class requires reference wrapping and the old code that didn't use reference wrapping will not work correctly as it will use copies in every recursive call.
Lambda functions	In AXE v.2 lambda functions used as rules and extractors are recognized via corresponding traits and no longer require `r_ref` and `e_ref` wrappers. The wrapper classes are retained for backward compatibility. Lambda functions in most cases should be polymorphic (auto parameter types or C++20 template lambda) to be recognized by trait class.
Parser functions	Many forwarding functions were eliminated due to the use of template class deduction guides. The rules look the same as before, but they no longer call the forwarding functions. The class names were changed to correspond to the previous function names. The new parser `axe::r_regex` to parse regular expressions was added. The parser uses `std::basic_regex` in implementation.
Variadic rules	AXE v.2 uses variadic templates for the composite types like `r_and<…>`, `r_or<…>`, etc. That provides for natural parse tree bindings and avoids deep nesting of types.
Parse tree functions	AXE v.2 introduced `parse_tree` function that returns a `result<Data, Iterator>::data` field which contains extracted data. There are additional convenience functions in `axe_utility.h` header file to simplify extraction of data and writing it in xml format.
Bug fixes and performance improvements	Several bugs were fixed in the code, as well as some shortcut rules. The extractors for integral types performance was improved.

Copyright © 2011-2026 GB Research, LLC.

Syntax Rules

AXE library uses overloaded C++ operators and functions to approximate EBNF syntax, providing additional facilities for convenience and improved performance. Most of AXE operator and function rules are iterator agnostic and can be used to parse text and binary data in files, containers, etc. The sequence and alternative rules (`&`, `|`, `-`) and the UTF-8 rules require at least forward iterators and check it with a `static_assert`, and `r_regex` requires bidirectional iterators. Input iterators, such as `std::istreambuf_iterator`, can be parsed through `r_buffered`. AXE functions and classes are defined in `axe` namespace, operators are in `axe::operators` inline namespace, and shortcuts are in `axe::shortcuts` namespace.

AXE operator rules

Syntax	Explanation
`A & B`	Match rule A and B in specified order
`A && B`	Match rule A and B in any order (same as `A & B | B & A`)
`A > B`	Match `A & B` atomically: once A matched, B must match, otherwise `axe::failure<charT>` is thrown with a message naming both rules (see `r_named`)
`A | B`	Match rule A or B in specified order
`A || B`	Match A followed by B, or A, or B (same as `A & B | A | B`)
`A ^ B`	Match permutations of A and B (same as `A & ~B | B & ~A`)
`A - B`	Match rule A but not B (same as `!B & A`)
`A % B`	Match A separated by B one or more times (same as `r_many(A, B)`)
`+A`	Match A one or more times (same as `r_many(A)`)
`*A`	Match A zero or more times (same as `r_many(A,0)`)
`~A`	Match A zero or one times (same as `r_many(A, 0, 1)`, but in a parse tree `~A` produces `std::optional` instead of `std::vector`)
`!A`	Match negation of A; this rule never advances iterators
`A * n` `n * A`	Match rule A exactly n times (same as `r_many(A, n, n)`)
`A/C`	New in AXE 2.1 Rule A is constrained by constraint C. If rule A is matched, then the Boolean constraint C is tested. If this test fails, then the whole composite rule also fails. C is called as `C()`, `C(first)`, `C(first, last)` or `C(first, last, end)`, where `[first, last)` is the range matched by A and `end` is the end of input.

Copyright © 2011-2026 GB Research, LLC.

A repetition (`*A`, `+A`, `A % B`, `A * n`, `r_many`) stops when an occurrence matches without consuming input, once the minimum number of occurrences is reached, because such an occurrence would match again forever. For example, `*(*_d)` and `*~_d` match empty input.

AXE Shortcuts

The following parser rules are included for convenience in namespace shortcuts:

Syntax	Explanation
`_`	Any character: `axe::r_any()`
`_a`	Single letter or _: `axe::r_alpha()`
`_d`	Decimal digit: `axe::r_num()`
`_double`	Double: `axe::r_double()`
`_hs`	Horizontal space: `axe::r_any(" \t")`
`_ident`	Identifier: `axe::r_ident()`
`_int`	Decimal integer: `axe::r_decimal()`
`_n`	New line: `axe::r_char('\n')`
`_o`	Octal digit: `axe::r_oct()`
`_r`	Carriage Return: `axe::r_char('\r')`
`_s`	Space separator: `axe::r_char(' ')`
`_t`	Tab character: `axe::r_char('\t')`
`_uint`	Decimal unsigned integer: `axe::r_udecimal()`
`_w`	Word: `axe::r_alnumstr()`
`_ws`	White space, one of `' '`, `'\t'`, `'\n'`, `'\r'`: `axe::r_pred(axe::is_wspace())`
`_x`	Hex digit: `axe::r_hex()`
`_z`	End of range: `axe::r_end()`
`_endl`	End of line: `*_hs & (_n | _z)`

AXE operator""

These convenience operators are included in namespace shortcuts:

Syntax	Explanation
`'c'_axe`	Creates `axe::r_char('c')` rule, for `char` and `wchar_t` literals

Copyright © 2011-2026 GB Research, LLC.

Syntax	Explanation
`"str"_axe`	With C++20 or later, creates `r_strlit<'s','t','r'>`, a rule whose characters are template arguments; `""_axe` creates `r_empty`, which matches without consuming input. It works with `char`, `wchar_t`, `char8_t`, `char16_t` and `char32_t` literals, and characters after an embedded `\0` are matched too. With C++17, creates `axe::r_str("str")`, for `char` and `wchar_t` literals.
`"abc"_any`	Creates `axe::r_any("abc")` rule, for `char` and `wchar_t` literals
`"abc"_regex`	Creates `axe::r_regex("abc")` rule, for `char` and `wchar_t` literals

For example,

```
using namespace axe::shortcuts;
auto assignment = _ident & *_ws & "="_axe & *_ws & _int & ';'_axe;
auto escaped = '\\' & "nrt\\"_any;
```

AXE functions and class constructors

In most cases the above operators and shortcuts are sufficient for creating general parsing rules. The functions and constructors described below provide additional facilities for creating less common rules or providing more flexibility to the above operators.

Syntax	Explanation

`r_expression(T& t)`
 	
`parse_expression(string, def_value)`
	`r_expression` is a rule that evaluates expressions with `+`, `-`, `*`, `/` and parentheses and stores result in provided variable `t`. If parameter `t` is a floating point value each operand of the expression is evaluated as a floating point. Otherwise the operands are evaluated as integer values. `r_expression` doesn't allow spaces in the expression string.
	`parse_expression` function accepts an expression string and returns the value of the longest leading expression, or `def_value` if the string doesn't start with an expression; the rest of the string is ignored. The white-space characters are skipped between all characters, so `"1 2"` is read as 12.

`r_many(A, B, min_occurrence = 1, max_occurrence = -1)`	Matches rule A separated by rule B from min_occurrence to max_occurrence times. Operators `*`, `+`, `%` create r_many rule.
`r_many(A, min_occurrence = 1, max_occurrence = -1)`	Matches rule A from min_occurrence to max_occurrence times
`r_select(A, B, C)`	Matches rule A, and if matched matches B, otherwise matches C (equivalent to `!A & C | A & B`, but A is evaluated only once)

Copyright © 2011-2026 GB Research, LLC.

Syntax	Explanation
`r_ref(A)`	Deprecated. Creates a reference rule wrapper for rule A. An `std::ref()` function can be used instead.
`r_find(A)`	Skip input until rule `A` matched. When matched it returns iterator range starting from the initial position to the end of matched rule. If you want to find the first match of rule `A` but leave the iterator at the beginning of that match, then you can use `r_find(!!A);`
`r_fail(F)`	Creates a rule to invoke function F on failure of the preceding rule (`A | r_fail(F)`). Allowed signatures: `void()`, `void(Iterator)`, `void(Iterator, Iterator)`, and `void(Iterator,Iterator,Iterator)`. With one argument the function receives the failed position; with two, the failed position and the end of input; with three, the position where the failed rule started, the failed position and the end of input. E.g. see example 1 below the table.
`r_fail(str = "r_fail")`	When the preceding rule fails `axe::failure<charT>` exception is thrown; its `what()` returns `str`, and its `message()` adds up to 39 characters of the input at the failed position (see AXE exceptions). The `charT` type depends on iterator type the rule was executed: `using charT = typename iterator_traits<Iterator>::value_type;`
`r_bool(b)`	Creates rule that evaluates boolean expression which becomes the result of match. The rule doesn't advance the iterators. `b` is a value convertible to bool, or a function returning a value convertible to bool and taking no arguments, the current position, or the current position and the end of input. A function taking iterators isn't called at the end of input, where the rule doesn't match. Since the argument is held by value, a variable needs a reference wrapper (`std::ref`) to be evaluated at parse time. For example, the following rule matches only numbers from 1 to 999 (see example 2 below the table).
`r_constrained(R,C)`	AXE 2.1 and later. Creates a constrained rule. Rule R is evaluated and if matched constraint C is tested. Constrained rule is matched if and only if rule R is matched and constraint C is satisfied. C has one of the signatures described for `A/C` above. To create a constrained rule, an `operator/` can be used. A constrained rule in many situations supersedes `r_bool`. For example, see example 3 below the table.
`r_empty()`	An empty rule always evaluates to true and doesn't advance the iterators

Copyright © 2011-2026 GB Research, LLC.

Syntax	Explanation
`r_char(c)`	Matches a single character `c`. In many composite rules characters are converted to `r_char`.
`r_bin(t)`	Matches a binary representation of standard layout type byte by byte. The input elements must be one byte each (e.g. `char`, `unsigned char`, `std::byte`).
`r_str(str)`	Matches a character string `str` (`const charT*` or `basic_string<charT>`). In many composite rules string literals are converted to `r_str`. `r_str(const charT*)` keeps the pointer, so the characters must outlive the rule: `r_str(s.c_str())` dangles once `s` changes, while `r_str(s)` copies the string.
`r_lit(a)`	This rule is a synonym for `r_char` (for `char` and `wchar_t`), `r_str` (for `const char*` and `const wchar_t*`), or `r_bin` (for other arithmetic types, including `char16_t` and `char32_t`) depending on type of parameter `a`. For example, `auto rule = r_lit('a') & r_lit("abc") & r_lit(123) & r_lit(1.23);`
`r_test(A)`	Matches rule A, but doesn't advance iterator (same as `!!A`)
`r_end()`	Matches end iterator, it is used to verify all input is consumed.
`r_var(t)`	Rule to assign to POD types. Reads from input `sizeof(t)` bytes and assigns to variable `t`
`r_array(a)`	`r_array(std::array<T, N>& a)` reads `N` values of the standard layout type `T` from binary input into `a`, `sizeof(T)` bytes each, like `r_var`. Fails if the input is shorter.
`r_range(iterator begin, iterator end)` `r_range(const Container& c)`	Matches range specified by begin and end iterators, or the elements of container `c`
`r_sequence(c, min_occurrence = 0, max_occurrence = -1)`	Reads up to `max_occurrence` values of `C::value_type` from binary input, `sizeof(C::value_type)` bytes each, and appends them to the container `c` with `push_back`. Matches if `c` then holds at least `min_occurrence` values. `c` is cleared when the rule is created, not on each parse, so parsing twice with the same rule appends to the values of the first parse.
`r_advance(size)`	Advances input by `size` elements. Fails if iterator range is shorter than `size` elements. `size` is copied when the rule is created; pass `std::ref(n)` to use the value `n` has when the rule is evaluated. This rule is usually used with negative rules to avoid matching the same rule multiple times. For example, see example 4 below the table.

Copyright © 2011-2026 GB Research, LLC.

Syntax	Explanation
`r_ident()`	Matches an identifier (`r_alpha() & *r_alnum()`)
`r_udecimal(t)` `r_udecimal()`	Matches unsigned decimal number, assigning value to t
`r_decimal(t)` `r_decimal()`	Matches signed decimal number, assigning value to t
`r_ufixed(t)` `r_ufixed()`	Matches unsigned fixed point number, assigning value to t
`r_fixed(t)` `r_fixed()`	Matches signed fixed point number, assigning value to t
`r_double(t)` `r_double()`	Matches floating point number, assigning value to t. Without an argument these rules only match. The signed rules accept spaces between the sign and the digits: `r_decimal(n)` reads `"- 42"` as -42.
`r_hex(t)`	Matches unsigned hexadecimal number, assigning value to t (`r_hex()` without an argument matches a single hexadecimal character)
`r_oct(t)`	Matches unsigned octal number, assigning value to t (`r_oct()` matches a single octal character)
`r_binary(t)`	Matches unsigned binary number (`'0'` and `'1'`), assigning value to t (`r_binary()` matches a single binary character)
`r_alpha()`	Matches a single alpha character ('A'-'Z','a'-'z', _); only ASCII letters are alpha characters
`r_alphastr()`	Matches a string of alpha characters
`r_alphastr(occurrence)`	Matches a string of alpha characters specified number of times
`r_alphastr(min_occurrence, max_occurrence)`	Matches a string of alphabetic characters specified number of times
`r_num()`	Matches a single digit character
`r_numstr()`	Matches a string of digits
`r_numstr(occurrence)`	Matches a string of digits specified number of times
`r_numstr(min_occurrence, max_occurrence)`	Matches a string of digits specified number of times
`r_alnum()`	Matches alpha-numeric character
`r_alnumstr()`	Matches a string of alpha-numeric characters
`r_alnumstr(occurrence)`	Matches a string of alpha-numeric characters specified number of times

Copyright © 2011-2026 GB Research, LLC.

Syntax	Explanation
`r_alnumstr(min_occurrence, max_occurrence)`	Matches a string of alpha-numeric characters specified number of times
`r_oct()`	Matches an octal character
`r_octstr()`	Matches a string of octal characters
`r_octstr(occurrence)`	Matches a string of octal characters specified number of times
`r_octstr(min_occurrence, max_occurrence)`	Matches a string of octal characters specified number of times
`r_hex()`	Matches hexadecimal character
`r_hexstr()`	Matches a string of hexadecimal characters
`r_hexstr(occurrence)`	Matches a string of hexadecimal characters specified number of times
`r_hexstr(min_occurrence, max_occurrence)`	Matches a string of hexadecimal characters specified number of times
`r_binary()`	Matches a binary character (`'0'` or `'1'`)
`r_binstr()`	Matches a string of binary characters
`r_binstr(occurrence)`	Matches a string of binary characters specified number of times
`r_binstr(min_occurrence, max_occurrence)`	Matches a string of binary characters specified number of times
`r_printable()`	Matches a printable character, an ASCII character from `' '` to `'~'`
`r_printablestr()`	Matches a string of printable characters
`r_printablestr(occurrence)`	Matches a string of printable characters specified number of times
`r_printablestr(min_occurrence, max_occurrence)`	Matches a string of printable characters specified number of times
`r_any()`	Matches any character; often used with `operator-` to create "any char except" rule. E.g. `*(_ - _s)` matches any characters except space.
`r_any(c1, c2)`	Matches any character in the range [c1, c2]
`r_any(str)`	Matches any character found in the string str
`r_anystr(occurrence)`	Matches a string of any characters specified number of times
`r_anystr(c1, c2)`	Matches a string of characters in the range [c1, c2]
`r_anystr(c1, c2, occurrence)`	Matches a string of characters in the range [c1, c2] specified number of times
`r_anystr(c1, c2, min_occurrence, max_occurrence)`	Matches a string of characters in the range [c1, c2] specified number of times

Copyright © 2011-2026 GB Research, LLC.

Syntax	Explanation
`r_anystr(str)`	Matches a string of characters found in the string str
`r_anystr(str, occurrence)`	Matches a string of characters found in the string str specified number of times
`r_anystr(str, min_occurrence, max_occurrence)`	Matches a string of characters found in the string str specified number of times
`r_pred(P)`	Matches a single character satisfying predicate P
`r_predstr(P)`	Matches a string of characters satisfying predicate P
`r_predstr(P, occurrence)`	Matches a string of characters satisfying predicate P specified number of times
`r_predstr(P, min_occurrence, max_occurrence)`	Matches a string of characters satisfying predicate P specified number of times
`r_named(R, const char* name)`	`r_named` rule assigns name to the rule R. This name is returned by `name()` function; `axe::operator>` uses this name to create an error message when the rule fails. E.g. `auto rule = r_lit("Hello") > r_named(r_lit("world"), "world rule");`
`r_ref(R)`	Rule reference wrapper is deprecated. The lambda functions can be automatically recognized as rules and no longer require wrapping. If you need to take a rule by reference (e.g. `r_rule<I>`) you can use `std::ref` function instead. This function is retained in this release for backward compatibility.

`r_regex(Args&&…)`
`r_regex(const CharT*)`
`r_regex(const basic_string&)`
	A parser for regular expressions. Parser implemented using `std::basic_regex`, parameters in the constructor are forwarded to the `std::basic_regex` constructor. The rule matches when the regular expression matches at the current position. Starting the expression with `^` makes failing matches faster, because `std::regex_search` then doesn't scan the rest of the input. Requires bidirectional iterators.

`r_depth_limit(R, max_depth)`	Limits the nesting depth of a recursive rule R; see Limiting recursion depth.
`r_utf8()` `r_utf8str()`	Match one, or one or more, well-formed UTF-8 encoded code points; see Validating UTF-8.

Example 1: `r_fail(F)`

```
A | r_fail([](auto i1, auto i2, auto i3)
{
   std::cerr << "parsing failed at position indicated by !\n"
```

Copyright © 2011-2026 GB Research, LLC.

```
   << std::string(i1, i2) << '!' << std::string(i2, i3);
});
```

Example 2: `r_bool(b)`

```
unsigned number = 0;
auto rule = _int >> number & r_bool([&] { return number > 0 && number < 1000;});
```

Example 3: `r_constrained(R,C)`

```
auto rule = _int /[](auto i1, auto i2)
{ auto number = get_as<int>(i1, i2); return number > 0 && number < 1000;};
```

Example 4: `r_advance(size)`

```
size_t s = 0;
// R is matched twice: by r_any() - R, then by R
*(r_any() - R) & R;
// R is matched once
*(r_any() - (R >> e_length(s))) & r_advance(std::ref(s));
```

AXE Iterator transformation rules

Transformation rules are used to modify the iterators to achieve specific purposes. The following iterator transformation rules are defined in AXE:

Syntax	Explanation

`r_skip(R, F)`
`r_skip(R, char)`
`r_skip(R, const char*)`
	Rule creates skip-iterator, which omits all sequence elements satisfying F (it can be a predicate or a rule) and passes skip-iterator to the rule R. When F denotes a predicate, it is applied to a single character from input and if it returns true the character is skipped. When F denotes a rule, all characters matching the rule are skipped.
	This rule is usually used to skip white spaces, e.g. using a predicate: `r_skip(R, axe::is_wspace());` or using a rule: `r_skip(R, axe::r_any(" \t\n\r"));`
	This is a convenience rule, that allows to avoid specifying characters, like spaces in rules explicitly, though specifying skip characters in rules explicitly is more flexible and may result in better performance. The characters are skipped between every pair of characters, not only between tokens: `r_skip(r_decimal(n), _ws)` reads `1 2` as 12.
	Overloads of `r_skip` function which accept characters and character strings create iterators skipping specified characters, e.g. the following rule skips all white-space characters: `r_skip(R, " \t\n\r");`
	The extractors in R receive the input without the skipped characters. A skipper rule that matches empty input, such as `*_ws`, stops skipping.

Copyright © 2011-2026 GB Research, LLC.

Syntax	Explanation
`r_ucase(string, locale = std::locale())`	This rule transforms iterator to upper case iterator, using specified locale (or default locale), and matches the string. The string itself isn't converted, so it must be written in upper case. It can be used for case insensitive match converting all characters to upper-case, e.g. `auto rule = _int & r_ucase("L");`
`r_lcase(string, locale = std::locale())`	Same as above, but uses lower case transformation; the string must be written in lower case.
`r_icase(string, locale = std::locale())`	Case insensitive match. It transforms both string and iterators to lower case. This rule is slower than `r_ucase` or `r_lcase`, because both input characters and string are converted.
`r_convert(R, F)`	Creates a rule that passes R an iterator whose elements are the input elements converted by function F. The result of F must be convertible to the iterator's `value_type`. The extractors in R receive the converted elements.
`r_buffered(R)`	Creates a buffer for input iterator, e.g. `istreambuf_iterator`. Inside the buffer R can backtrack like any rule. Input iterators cannot be rolled back, so if a buffered rule is not matched an `axe::failure<charT>` exception is thrown with the message `"buffered rule failed"`. For the same reason R must end at the end of input or where it stopped reading. A rule that reads past its match, as `_int` reads the character after the number to see that it isn't a digit, throws `axe::failure<charT>` with the message `"input_buffer: rollback not possible"` unless the match is at the end of input; `_int & ' '` consumes that character. The returned position is in the input, so the next parse continues there. To minimize the amount of copying and the size of the buffer, only wrap the minimum necessary number of atomic rules in `r_buffered`.

`r_rule` inside rules that change the iterator type

`r_skip`, `r_convert` and `r_buffered` don't pass the input iterator to their sub-rule; they pass `skip_iterator<I, F>`, `convert_iterator<I, F>` and `input_buffer<I>::iterator` respectively. An `r_rule` used in the sub-rule, including through `std::ref`, must be declared with that iterator type, otherwise compilation fails at the `static_assert` in `r_rule::operator()`:

```
std::string text = "abc [ abc [ 2 ] ]";
using I = std::string::iterator;
// the iterator created by r_skip(..., _ws)
axe::r_rule<axe::skip_iterator<I, axe::r_pred<axe::is_wspace>>> r;
auto a = axe::r_lit("abc") & std::ref(r);
r = '[' & (_int | a) & ']';
bool matched = axe::parse(axe::r_skip(a, _ws), text).matched;
```

Copyright © 2011-2026 GB Research, LLC.

For `r_skip`, `F` must be exactly the skipper type it stores: the decayed type of a predicate or rule (`std::decay_t<decltype(_ws)>`, not `decltype(_ws)`), `is_char_t<C>` for a character, and `is_any_t<const C*>` for a string. Such a rule works only inside that transformation. A function object with a templated `operator()`, like `r_expression` in `axe_expression.h`, works with any iterator type. Alternatively, keep `r_rule<I>` and skip white space explicitly with `*_ws`.

AXE Predicate Functions

The predicates are types: for example, `is_alpha()` creates an object of type `axe::is_alpha`.

Syntax	Explanation
`is_alpha()`	Returns true for single alpha character (an ASCII letter or '_').
`is_num()`	Returns true for single decimal digit
`is_alnum()`	Returns true for single alpha or digit character
`is_hex()`	Returns true for single hex character
`is_oct()`	Returns true for single oct character
`is_bin()`	Returns true for single binary character (`'0'` or `'1'`)
`is_printable()`	Returns true for single printable character (an ASCII character from `' '` to `'~'`)
`is_space()`	Returns true for single space character (`' '`)
`is_wspace()`	Returns true for a single white-space character (`" \t\n\r"`)
`is_char(c)`	Returns true if input matches character c
`is_any(c1, c2)`	Returns true if input matches any character in the range [c1, c2]
`is_any(const char* str)`	Returns true if input matches any character from the string
`is_any()`	Always returns true.

AXE Predicate Operators

Operators called on predicates return the following composite predicates:

Syntax	Explanation
`P1 && P2`	Evaluates to true if P1 and P2 evaluate to true
`P1 || P2`	Evaluates to true if P1 or P2 evaluate to true
`P1 ^ P2`	Evaluates to true if P1 or P2 evaluate to true, but not both
`!P`	Evaluates to true if P evaluates to false

Copyright © 2011-2026 GB Research, LLC.

AXE polymorphic rule

AXE library provides polymorphic type `axe::r_rule<Iterator>` for recursive rule definitions and runtime parser creation. It uses type erasure to assign any other parser rule to it. Polymorphic rule is often used in recursive parser definitions. Since AXE v.2.0 (unlike previous versions) when creating composite rules the sub-rules are copied or moved, so you would need to create a reference wrapper for recursive parsers, e.g.

```
axe::r_rule<Iterator> r;
auto a = "abc"_axe & std::ref(r);
r = '[' & (_int | a) & ']';
```

A default-constructed `r_rule` matches empty input, and converts to `false` until a rule is assigned to it. `r_rule<Iterator>` can only be called with iterators convertible to `Iterator`; see `r_rule` inside rules that change the iterator type.

Limiting recursion depth

Recursive grammars (defined with `r_rule` and `std::ref`) recurse on the C++ stack, so deeply nested input, such as 100,000 `[` characters fed to a JSON grammar, can overflow the stack. `r_depth_limit` (`axe_depth.h`) wraps a rule and bounds its nesting depth:

```
axe::r_rule<I> value;
auto array = '[' & *_ws & (']'_axe | (*_ws & std::ref(value) & *_ws) % ',' & ']');
value = axe::r_depth_limit(json_string | _double | array /* | ... */, 256);

try
{
    axe::parse(std::ref(value), begin, end);
}
catch (const axe::depth_limit_exceeded<I>& ex)
{
    // ex.position() - where the rejected invocation started
    // ex.max_depth() - the limit that was exceeded
}
```

- `r_depth_limit(rule, max_depth)` returns `r_depth_limit_t<Rule>`. Each invocation of the wrapper increments the depth for the duration of the match. When the depth exceeds `max_depth`, the wrapper throws `axe::depth_limit_exceeded<Iterator>` before invoking the wrapped rule. The outermost invocation has depth 1, and `max_depth()` returns the limit. In a parse tree the wrapper produces the data of the wrapped rule.
- `depth_limit_exceeded<Iterator>` derives from `axe::failure<value_type>`, so existing handlers of `failure` also catch it. Catch it first to tell a depth violation apart from a syntax error.
- Every invocation counts, including one that fails to match. `'(' & ~r & ')'` invokes `r` one level deeper than it matches; `'(' & (r_char(')') | r & ')')` doesn't.

Copyright © 2011-2026 GB Research, LLC.

- The depth is kept per wrapper object and per thread. The same rule object can be used by several threads concurrently. Nested parses of the same rule on one thread (for example, started from a semantic action) are counted cumulatively. The count is restored on every exit path, including exceptions thrown by semantic actions.
- Put the wrapper at a single point that every recursive cycle passes through, typically the right-hand side of the `r_rule` assignment. Separate copies of a wrapper count separately.
- A parse must not be suspended (for example, by a fiber or coroutine switch) while inside the wrapper.
- Choose `max_depth` so that `max_depth` levels fit the stack of the parsing thread. Stack usage per level depends on the grammar and the build. For the JSON grammar in `test/jason_test.cpp` it is about 40 KB per nesting level in an MSVC Debug build and about 3 KB in Release.

Validating UTF-8

`axe_utf8.h` provides terminal rules for well-formed UTF-8 (RFC 3629, Unicode Table 3-7):

- `r_utf8()` matches exactly one well-formed UTF-8 encoded code point. It rejects stray continuation bytes, overlong encodings, surrogate code points U+D800..U+DFFF encoded directly, code points above U+10FFFF, the bytes 0xC0, 0xC1 and 0xF5..0xFF, and truncated sequences. On failure the returned position is the start of the ill-formed sequence.
- `r_utf8str()` matches one or more well-formed code points and stops before the first ill-formed sequence or at the end of input.
- Both rules work with forward (multi-pass) iterators over `char`, `signed char`, `unsigned char` and `char8_t`. Single-pass input iterators such as `std::istreambuf_iterator` are rejected at compile time: read such input into a buffer first.

```
// the whole input is UTF-8
bool valid = axe::parse(*axe::r_utf8() & axe::r_end(), text).matched;
// an unescaped JSON character
auto json_char = axe::r_utf8() - '"' - '\\' - axe::r_any('\x00', '\x1f')
    | '\\' & json_escaped;
```

AXE Parse Tree bindings

When `axe::parse_tree` function is used for a composite rule, it returns in `axe::result<D,I>::data` field the combination of the following data types:

Syntax	Explanation
`A & B & …`	`std::tuple<DA, DB, …>` You can use `std::get<N>` function to extract individual data fields.
`A | B | …`	`std::variant<DA, DB,…>` You would normally use `std::visit` function to extract individual data.

Copyright © 2011-2026 GB Research, LLC.

Syntax	Explanation
`*A`, `+A`, `A % B`, `r_many(A,…)`	`std::vector<DA>` The separator rules are not extracted.
`A - …`	`std::tuple<…, DA>` The last element of the tuple is the extracted data from rule A.
`~A`	`std::optional<DA>`
`r_find(A)`, `r_named(A, name)`, `A/C`, `r_depth_limit(A, n)`	`DA` -- the same type as for rule A.
`A >> E`	`axe::it_pair<I>`, and the extractor E is invoked as with `parse`. If `AXE_ALLOW_PARSE_TREE_EXTRACTION` is defined before `axe.h` is included, `DA`, which is also passed to E as its only argument; every extractor of a rule passed to `parse_tree` must then accept the parse tree data.
`A && B`, `A || B`, `A ^ B`, `A > B`, `!A`, `r_test(A)`, `r_select(A, B, C)`, the iterator transformation rules, and all other rules, including custom rules that do not provide parse tree extraction	`axe::it_pair<I>`, which holds begin and end iterators of parsed sequence.

AXE Utility functions

The following functions are defined in the axe namespace in `axe_utility.h` (`parse_expression` in `axe_expression.h`):

Syntax	Explanation
`template<class R, class Txt>` `auto parse(R&& r, Txt&& txt)`	Function applies the rule `r` to the container `txt` and returns `result<Iterator>`.
`template<class R, class I>` `auto parse(R&& r, I begin, I end)`	Same as the above, but the function takes the iterators `begin` and `end`. An alternative is to call `operator()` on the rule directly: `r(begin, end);`
`template<class R, class Txt>` `auto parse_tree(R&& r, Txt&& txt)`	Function applies the rule **r** to the container **txt** and returns `result<Data, Iterator>`. The field `result<Data, I>::data` contains the parse tree for this operation. See AXE Parse Tree bindings above.
`template<class R, class I>` `auto parse_tree(R&& r, I begin, I end)`	Same as the above, but the function takes the iterators `begin` and `end`.

Copyright © 2011-2026 GB Research, LLC.

Syntax	Explanation
`template<class D, class Data>` `auto get_as(const Data& data)`	Function uses AXE extractors to convert `result<Data, I>::data` field passed as an argument to specified type `D`. Function can convert tuple, variant, vector, optional, it_pair data types that comprise parse-tree data: the text from the beginning of the first to the end of the last element is converted. AXE extractors provide converters to the arithmetic types, string types, and a stream based converter (see Extracting into variables). If none of the AXE extractors can convert the parse-tree data to the requested type D a compilation error will be reported.

`get_as<D>(I first, I last)`
`get_as<D>(const it_pair<I>& range)`
`get_as<D>(const result<I, T>& res)`
`get_as<D>(const result<I, T>& res, Fn fn)`
	The same conversion of a range of iterators, of an `it_pair`, of the `data` of a parse result, and of `fn(res.data)`. An empty range gives a value-initialized `D`.

`template<class R>` `inline auto get_name(const R& r)`	Returns name if it's a named rule. Otherwise returns `type_info::name`. Named rules is a debugging convenience.
`template<class T, class C, class Traits, class Alloc>` `inline auto parse_expression(const std::basic_string<C, Traits, Alloc>& str, T def_value)`	Function parses the expression in string `str` using `axe::r_expression` parser. Returns parsed value of type `T` if successful. Otherwise returns `def_value`. See `r_expression` above.
`auto write_xml(const parse-tree-data-type& v, std::ostream& os, size_t level = 0)`	Function writes `result<Data, I>::data` field passed as argument to the stream in xml format. Parameter `level` is used to create indentation: each nesting level is indented by one space. The elements are `<tuple size="n">`, `<variant index="i">`, `<vector size="n">` (`<vector size="0"/>` when empty) and `<optional matched="true">` (`<optional matched="false"/>` when empty), and the parsed text is written as `<![CDATA[parsed-text]]>`. For example, see the `write_xml` example below the table.

Example: `write_xml`

```
axe::write_xml(axe::parse_tree(*axe::r_find(".o."_regex),
    "The quick brown fox jumps over the lazy dog").data, std::cout);
```

the output:

Copyright © 2011-2026 GB Research, LLC.

```
<vector size="4">
 <![CDATA[row]]>
 <![CDATA[fox]]>
 <![CDATA[ ov]]>
 <![CDATA[dog]]>
</vector>
```

AXE exceptions

`axe::failure<charT>` (`axe_exception.h`), derived from `std::exception`, is thrown by `r_fail(str)`, by `A > B` when B doesn't match, by `r_buffered` when its rule doesn't match, and by `axe::throw_failure`. `what()` returns the message; `message()` returns a `std::basic_string<charT>` with the message and up to 39 characters of the input at the failed position. `charT` is the `value_type` of the iterator.

`axe::depth_limit_exceeded<Iterator>`, derived from `failure`, is thrown by `r_depth_limit`; see Limiting recursion depth.

Creating custom predicates

Custom predicate is a class or lambda function which defines `operator()`, taking a parameter of character type and returning `bool`. Predicates are usually used with `r_pred` and `r_predstr` function to create corresponding rules.

For `custom` class to be considered a predicate the trait `axe::is_predicate_v<custom>` must be true: `custom` must return `bool` for an argument of type `char`, `signed char`, `unsigned char`, `wchar_t`, `char16_t` or `char32_t`.

For example, `axe::is_num` is a predicate that matches a single digit:

```
struct is_num
{
    bool operator()(char c) const { return c >= '0' && c <= '9'; }
};
```

Lambda functions can also be used to create a predicate. For example,

```
auto is_num = [](auto c) { return c >= '0' && c <= '9'; };
auto number_rule = axe::r_pred(is_num);
auto numbers_rule = axe::r_predstr(is_num);
```

Creating custom rules

AXE library provides many facilities to create parsing rules using shortcuts, operators, and lambda functions, so you almost never need to create a custom rule class. Nevertheless, if such need arises, this section explains how you can create a custom rule class.

Copyright © 2011-2026 GB Research, LLC.

Custom rule is a class or lambda function which defines const `operator()`. There are two supported overloads of this operator, one of which creates parse tree and the other doesn't. The composite rules call their operands as const: `&`, `-` and `|` report an operand without a const `operator()` by a `static_assert` with the message "every operand must be a rule with a const operator()", the other rules by a compile error.

The simplest form of `operator()`, which doesn't create parse tree, takes a pair of iterators and returns `axe::result<Iterator>` class.

If custom rule is extracting a parse tree then it must additionally define `operator()` which takes `axe::it_pair` parameter and returning `axe::result<Data, Iterator>` type. The data field in the return type should contain extracted value of one of supported types (see: AXE Parse Tree bindings).

For `custom` class to be considered a rule the trait `axe::is_rule_v<custom>` must be true. Automatic rule detection is limited to a few predefined iterator types, so in general case `operator()` should be a template to be recognized as a rule. In some cases the return type of `operator()` must be explicit `axe::result<Iterator>` and not `auto` or `decltype(auto)`. The reason for that are the rule traits, which may not work with `auto` return type. Compiler forces instantiation of the functions with `auto` return type which can result in hard error instead of SFINAE.

E.g. custom rule matching new line:

```
class custom
{
public:
    template<class Iterator>
    axe::result<Iterator> operator()(Iterator i1, Iterator i2) const
    {
        // match new line
        bool matched = i1 != i2 && *i1 == '\n';
        // return result, advancing iterator if matched
        return axe::result(matched, matched ? std::next(i1): i1);
    }
};

void test()
{
    static_assert(axe::is_rule_v<custom>, "Error: Custom is not a rule");
}
```

Lambda functions can also be used as rules. They must take two auto parameters (or C++20 lambda template) to be recognized as rules.

E.g.

```
auto custom = [](auto i1, auto i2)
{
    bool matched = i1 != i2 && *i1 == '\n';
    return axe::result(matched, matched ? std::next(i1) : i1);
};
```

Copyright © 2011-2026 GB Research, LLC.

Recursive rules

AXE being a recursive descent parser doesn't allow left recursion[2]. But the right recursion in custom rules is possible.

When writing template `operator()`, it can be inlined in the body of the rule class. The dependent names in `operator()` of the rule are looked up at the point of instantiation. After forward declaring a custom rule, the name can then appear in the template `operator()` of other rules before the type of that custom rule is complete.

For example, the AXE rule for `r_expression` is forward declared and the name is used in `r_group_t` rule and called recursively:

```
template<class T> struct r_expression;

template<class T>
struct r_group_t
{
    // skipped ...
    template<class It>
    result<It> operator() (It i1, It i2) const
    {
        return ('(' & r_expression<T>(value_) & ')')(i1, i2);
    }
};

template<class T>
struct r_factor_t
{
    // skipped ...
    template<class It>
    result<It> operator() (It i1, It i2) const
    {
        return (r_decimal(value_) | r_group_t<T>(value_))(i1, i2);
    }
};

template<class T>
struct r_term_t
{
    // skipped ...
    template<class It>
    result<It> operator() (It i1, It i2) const
    {
        return (r_factor_t<T>(value_) & // skipped …
        ))(i1, i2);
    }
};

template<class T>
struct r_expression
{
    // skipped ...
    template<class It>
```

Copyright © 2011-2026 GB Research, LLC.

```
    result<It> operator() (It i1, It i2) const
    {
        return
        (r_term_t<T>(value_) & // skipped ...
        )(i1, i2);
    }
};
```

For details of `r_expression` implementation see axe_expression.h file. There is also a convenience function defined in this file, which parses expression and returns value if successful or `def_value` if parsing failed:

```
template<class T, class C, class Traits, class Alloc>
inline auto parse_expression(const std::basic_string<C, Traits, Alloc>& str, T
def_value)
```

Though lambda functions do not allow recursive definition[3], standard polymorphic functions (`std::function`) can be used recursively for rule definition. For example,

```
std::function<axe::result<const char*> (const char*, const char*)> block
    = [&](const char* i1, const char* i2)
{ return (';'_axe | '{' & std::ref(block) & '}')(i1,i2); };
```

AXE provide a polymorphic rule `axe::r_rule`, which can also be used to define recursive rules. One thing to remember is that composite rules keep sub-rules by value, so `axe::r_rule` must be wrapped in `reference_wrapper` using `std::ref` function (`std::reference_wrapper` is recognized as a rule) in order to refer to the same function object.

For example,

```
axe::r_rule<I> json_value;
auto json_array = json_spaces & '['
    & axe::r_many(json_spaces & std::ref(json_value) & json_spaces, ',', 0)
    & json_spaces & ']';
json_value = json_string | axe::r_double(d) | json_object | json_array
    | "true" | "false" | "null";
```

A recursive rule can overflow the stack on deeply nested input; `r_depth_limit` bounds the nesting depth.

---

2. Left recursion: Wikipedia
3. You can define lambda taking another lambda as a parameter, then proving the same lambda as the parameter, thus creating recursion. This usually would fail when creating parsing rules though, because compiler wouldn't be able to deduce the return type for such lambda causing rule traits to fail, and providing the return type explicitly is impossible in most situations.

Copyright © 2011-2026 GB Research, LLC.

Semantic Actions

There are two ways to extract data from a parser. One is to use `parse_tree` function and get the whole parse tree in `axe::result<Data, Iterator>::data` field. The other way is using an in-place extractor, which creates an extractor rule. Both approaches have their pluses and minuses.

AXE in-place extractors

Syntax	Explanation
`A >> E`	Operator >> creates an extractor rule. If rule A is matched, the iterator range for that rule is passed to extractor E. The extractor can accept zero, one, two, or three arguments of Iterator type. If the extractor accepts one argument then the iterator pointing past the end of the matched range will be passed to the function. If the extractor accepts two arguments, the beginning and the end of the matched rule are passed. If the extractor accepts three arguments then in addition to previous a third iterator to the end of the input sequence is passed. Extractors can be chained, each one receiving the matched range of the rule. They are invoked in the same order as the associativity of the operators, so a later invoked extractor can use the result of the previously executed one. For example, the following rule will extract and print the value and length of an identifier (see the `A >> E` example below the table).
`A >> t`	If `t` is a variable that isn't an extractor, the matched text is converted and assigned to `t`; see Extracting into variables.
`A >> e_ref(E)`	Extractor reference wrapper is deprecated. The lambda functions can be automatically recognized as extractors and no longer require wrapping. This function is retained for backward compatibility.
`A >> e_length(t)`	Extractor assigns length of matched range to t.
`A >> e_push_back(c)`	Extractor converts each matched range to `C::value_type`, as `A >> t` does, and appends it to `c` with `emplace_back`, e.g. to a `std::vector<std::string>` or a `std::vector<int>`.

Copyright © 2011-2026 GB Research, LLC.

Syntax	Explanation
`r_fail(F)`	`r_fail(F)` is a wrapper for a function F, which is called when the preceding rule has failed. The function F can accept zero, one, two, or three arguments. If the function accepts one argument the iterator pointing to the failed position is passed to it. If the function accepts two arguments the iterator pointing to the failed position and the iterator to the end of the sequence are passed to it. If the function accepts three arguments the iterator pointing to the beginning of the sequence, the iterator pointing to the failed position, and the iterator to the end of the sequence are passed to it. This function is always used with operator `|` to create a rule. For example, see the `r_fail(F)` example below the table.

Example: `A >> E`

```
std::string name;
size_t length(0);

auto rule = r_ident() >> name >> e_length(length);

parse(rule, "an_identifier"s);
std::cout << "parsed identifier: " << name
<< " is " << length << " characters long";
```

Example: `r_fail(F)`

```
auto on_fail = []
{
   std::cerr << "Error: expecting identifier";
};

auto rule = r_ident() | r_fail(on_fail);
```

Extracting into variables

`A >> t`, where `t` is a variable that isn't an extractor, converts the text matched by A according to the type of `t`. `e_push_back` and `get_as` use the same conversions.

Type of `t`	Value assigned
`std::basic_string`	the matched text
`std::basic_string_view`	a view of the matched text, which refers to the input; with C++20 for any contiguous input, with C++17 only for pointers to the input
`char`, `signed char`, `unsigned char`, `wchar_t`; `std::byte` (from `std::byte` input)	the first matched element

Copyright © 2011-2026 GB Research, LLC.

Type of `t`	Value assigned
`short`, `unsigned short`, `int`, `unsigned`, `long`, `unsigned long`, `long long`, `unsigned long long`	the number converted with `std::stoi`, `std::stoul`, `std::stol`, `std::stoll` or `std::stoull`, which throw `std::invalid_argument` when the text isn't a number
`float`, `double`, `long double`	the number converted with `std::stof`, `std::stod` or `std::stold`
`bool`	whether the match is non-empty
`std::basic_ostream`	the matched text is written to the stream, e.g. `rule >> std::cout`
any other type	read from the matched text with `operator>>`

Creating custom semantic actions

Custom semantic action is a class which defines const `operator()`, taking zero, one, two, or three iterators and returning `void`. For custom class to be considered an extractor the trait `axe::is_extractor_v<custom>` must be true. In most cases the `operator()` must be a template to be recognized by the trait class. The extractor rule calls it as `const`, so state that it changes must be held by reference, as `count_` is below.

E.g. custom extractor counting new lines:

```
class custom
{
    size_t& count_;
public:
    custom(size_t& count) : count_(count) {}
    template<class Iterator>
    void operator()(Iterator i1, Iterator i2) const
    {
        if(i1 != i2 && *i1 == '\n') ++count_;
    }
};

void test()
{
    static_assert(axe::is_extractor_v<custom>,
                  "Error: Custom is not an extractor");
}
```

Lambda functions can also be used as semantic actions. They must take auto parameters to be recognized by `axe::is_extractor_v` trait.

E.g.

```
size_t count = 0;
auto custom = [&](auto i1, auto i2)
{
```

Copyright © 2011-2026 GB Research, LLC.

```
    if(i1 != i2 && *i1 == '\n') ++count;
};
using namespace axe::shortcuts;
parse(*(_ >> custom), "count\neol\nin this text\n");
```

The traits test extractors (and rules) with the pointer iterator types `char*`, `const char*`, `wchar_t*` and so on, so a lambda with a deduced return type is instantiated with those types. A lambda body that compiles only for the iterator type used in the parse, e.g. one that subtracts `text.begin()`, causes a hard error inside the traits; declaring the return type (`[&](auto i1, auto i2) -> void { … }`) avoids that.

Using parse tree

In addition to in-place extractors AXE can create a parse tree for parser.

In-place extractors are easy to use and produce faster parsers. But they become part of the rule, reducing the flexibility of using the rule in different contexts. They also practically always bind to l-values receiving the result. You must be careful not to return such parsers from (lambda) functions, because the references will become dangling when the l-values go out of scope.

The alternative way of extracting data from a parse result is using a parse tree. The parse tree is returned as a data field in `axe::result<Data, Iterator>` struct. This field is created based on the rules described in AXE Parse Tree bindings table.

When you call `axe::parse_tree` function the data field will contain the parse tree. That allows to separate the rules from data extraction, so the rules can be reused in different context requiring different extraction. It also simplifies returning rules from (lambda) functions because they don't have to deal with possibility of dangling references.

The cost you pay for using the parse tree is slower execution because during the parsing process the parse tree data keeps growing and is returned on the function stack. It's also more difficult to reason about the extractor as you must correctly recognize the bindings by looking at the rule. If you do it incorrectly compiler will likely stop you with a long error message. Another inconvenience of parse tree is that it makes the rule less flexible to changes, because parse tree extractor and the rule must be modified in sync.

To simplify the process of extraction from the parse tree, AXE provides `get_as` function template:

```
template<class D, class Data>
auto get_as(const Data& data);
```

- where `D` is the type you want to convert to, and `data` is the `result<Data, Iterator>::data` field returned from the `parse_tree` function, or any part of it.

This function is overloaded for all known parse tree data types and can convert to arithmetic, string, and other stream-able types using the same extractor functions that are used for in-place extraction (see AXE Utility functions for the other overloads).

You can also write an output in xml format for the whole parse tree by using the `write_xml` function:

Copyright © 2011-2026 GB Research, LLC.

```
auto write_xml(const parse-tree-data& data, std::ostream& os, size_t level = 0);
```

The xml tags used correspond to the data type of the node in the parse tree: `<tuple>`, `<variant>`, `<vector>`, `<optional>`, and the parsed text is written as `<![CDATA[parsed-text]]>`

See Examples section for ways of using parse tree extraction.

Copyright © 2011-2026 GB Research, LLC.

Examples

Examples below omit the include files and the using declarations:

```
#include <axe.h>
#include <algorithm>
#include <fstream>
#include <functional>
#include <iostream>
#include <map>
#include <numeric>
#include <set>
#include <stdexcept>
#include <string>
#include <tuple>
#include <vector>

using namespace axe;
using namespace axe::shortcuts;
```

Zip code parser

Zip codes are a system of postal codes used by the United States Postal Service (USPS) since 1963. The basic format consists of five decimal numerical digits. An extended ZIP+4 code, introduced in the 1980s, includes the five digits of the ZIP code, a hyphen, and four more digits that determine a more precise location than the ZIP code alone.[4]

Therefore, we can write the following rule to match a ZIP code:

```
auto zip_rule = _d * 5 & ~('-' & _d * 4) & !_d;
```

where:

- `_d * 5` expression creates a rule that matches exactly 5 digits
- `~('-' & _d * 4)` expression creates a rule that optionally matches '-' followed by 4 digits
- `!_d` expression creates a rule that test the next character without advancing the iterator, and matches everything except a digit.

The following rule matches (and skips) any portion of the text that is not zip_rule: `*(_ - zip_rule)`

To skip any portion of the text that is not zip_rule and then extract string from zip_rule, we can write this rule: `*(_ - zip_rule) & zip_rule >> [](auto i1, auto i2) {…}`

The following function extracts and prints zip codes found in the text:

```
void print_zip(const std::string& text)
{
    // rule to match zip code
    auto zip_rule = _d * 5 & ~('-' & _d * 4) & !_d;
```

Copyright © 2011-2026 GB Research, LLC.

```
    // skip everything that is not zip_rule and extract zip_rule
    auto text_rule = *(*(_ - zip_rule) & zip_rule
        >> [](auto i1, auto i2)
    {
        std::cout << std::string(i1, i2) << "\n";
    });

    parse(text_rule, text);

    std::cout << "Now using the parse tree\n";

    // the same parse rule without the in-place extractor
    auto text_rule1 = *(*(_ - zip_rule) & zip_rule);

    // this is an example of using a parse tree to extract data
    // the rule above produces vector of tuple,
    // we are only interested in the second value in that tuple corresponding to
zip_rule
    for (auto& d : parse_tree(text_rule1, text).data)
    {
        std::cout << axe::get_as<std::string>(std::get<1>(d)) << "\n";
    }
}

// a function to run a test of the parser rules created above

void test_zip()
{
    std::cout << "------------------------------------------------------------test_zip:\n";
    auto zips = std::string(R"*(94302 Zip Code
94303-1011 Zip Code
94309 Zip Code
94301 Zip Code
94304 Zip Code
94306 Zip Code
94307)*"
);
    print_zip(zips);
    std::cout << "-----------------------------------------------------------------------\n";
}
```

4. ZIP code: Wikipedia

Telephone number parser

The traditional convention for phone numbers [in United States] is (AAA) BBB-BBBB, where AAA is the area code and BBB-BBBB is the subscriber number. The format AAA-BBB-BBBB or sometimes 1-AAA-BBB-BBBB is often seen. Sometimes the stylized format of AAA.BBB.BBBB is seen.[5]

We can write the following four rules corresponding to these conventions:

Copyright © 2011-2026 GB Research, LLC.

```
// four common telephone formats
auto tel1 = '(' & _d * 3 & ')' & _d * 3 & '-' & _d * 4;
auto tel2 = _d * 3 & '-' & _d * 3 & '-' & _d * 4;
auto tel3 = "1-" & _d * 3 & '-' & _d * 3 & '-' & _d * 4;
auto tel4 = _d * 3 & '.' & _d * 3 & '.' & _d * 4;
```

The following function prints telephone numbers found in the text. As a demonstration of constrained rule, a list of numbers, which don't satisfy the specified constraint, is excluded. A constraint lambda function checks the set of excluded numbers and returns true if a number is not excluded.

```
//------------------------------------------------------------------------------
void print_tel(const std::string& text)
{
    // four common telephone formats
    auto tel1 = '(' & _d * 3 & ')' & _d * 3 &'-' & _d * 4;
    auto tel2 = _d * 3 & '-' & _d * 3 & '-' & _d * 4;
    auto tel3 = "1-" & _d * 3 & '-' & _d * 3 & '-' & _d * 4;
    auto tel4 = _d * 3 & '.' & _d * 3 & '.' & _d * 4;

    // exclude known fax numbers
    const std::set<std::string> excluded_numbers{"408-278-7444", "650-617-3120"};

    auto tel2_filtered = tel2 / [&](auto i1, auto i2)
    {
        return excluded_numbers.count(std::string(i1, i2)) == 0;
    };

    auto tel = tel1 | tel2_filtered | tel3 | tel4;
    // extracting rule
    auto text_rule = *(*(_ - tel) & tel
        >> [](auto i1, auto i2)
    {
        std::cout << std::string(i1, i2) << "\n";
    });

    parse(text_rule, text);

    std::cout << "Using parse_tree function:\n";

    // using parse tree for data extraction
    for (auto p : parse_tree(*(*(_ - tel) & tel), text).data)
    {
        std::cout << get_as<std::string>(std::get<1>(p)) << "\n";
    }
}

void test_tel()
{
    std::cout << "------------------------------------------------------------test_tel:\n";
    auto tels = std::string(R"*(call City Hall at (650)329-2100
Police Desk at 329-2406
Phone: 650-329-2258, Fax: 650-617-3120
1505 Meridian, San Jose, CA
Phone: 408-278-7400
Fax: 408-278-7444
```

Copyright © 2011-2026 GB Research, LLC.

```
94307)*");
    print_tel(tels);
    std::cout << "----------------------------------------------------------------------\n";
}
```

The fax numbers 650-617-3120 and 408-278-7444 are excluded by the constraint:

```
------------------------------------------------------------test_tel:
(650)329-2100
650-329-2258
408-278-7400
Using parse_tree function:
(650)329-2100
650-329-2258
408-278-7400
----------------------------------------------------------------------
```

5. Local conventions for writing telephone numbers: Wikipedia

CSV parser

This example shows how to create a parser for comma-separated values (csv) format[6]. In this example we allow any printable characters (except comma character) inside the values. Additionally, we allow spaces in the beginning and the end of each value, which are removed during parsing. The spaces inside string values are preserved. The program below prints each extracted value in angle brackets:

```
<Year><Make><Model><Trim><Length>
<2010><Ford><E350><Wagon Passenger Van><212.0>
<2011><Toyota><Tundra><CREWMAX><228.7>
```

```
auto csv(const std::string& text)
{
    // comma separator including trailing spaces
    auto cvs_separator = *_hs & ',';
    // rule for comma separated value
    auto csv_value = *_hs & +(_ - cvs_separator - _endl)
        >> [](auto i1, auto i2)
    {
        std::cout << "<" << std::string(i1, i2) << ">";
    };

    // rule for single string of comma separated values
    auto line = csv_value % cvs_separator
        & _endl >> [] { std::cout << "\n"; };

    // file contaning multiple csv lines
    auto csv_file = +line & _z
        | axe::r_fail([](auto i1, auto i2)
        {
            std::cout << "\nFailed: " << std::string(i1, i2);
        });
```

Copyright © 2011-2026 GB Research, LLC.

```
    parse(csv_file, text);
}

void test_cvs()
{
    std::cout << "------------------------------------------------------------test_cvs:\n";
    auto text = std::string(R"*(Year, Make, Model, Trim, Length
2010,Ford,E350, Wagon Passenger Van ,212.0
2011 , Toyota, Tundra, CREWMAX, 228.7 )*");
    csv(text);
    std::cout << "------------------------------------------------------------------\n";
}
```

6. Comma-separated values: Wikipedia

INI File parser

INI file was once popular format for configuration files on the Windows and other platforms[7].

This example demonstrates how one can create a parser for INI file, which parses properties, sections, and comments.

Simplified EBNF rule for INI file is this:

```
ini_file ::= {comment}* {section {property | comment}*}*
```

The following function parses INI record, specified by a pair of iterators, and outputs INI records converted to xml.

```
template<class I>
void ini(I begin, I end)
{
    using namespace axe::shortcuts;

    auto close_section = [] (const std::string& section)
    {
        if (!section.empty())
        {
            std::cout << "</" << section << ">\n";
        }
    };

    std::string open_section;
    std::string section_name;

    // section rule, can end with traling spaces
    auto section = *_hs & ('[' & _ident >> section_name & ']') >> [&]
    {
        close_section(open_section);
        open_section = section_name;
        std::cout << "<" << open_section << ">\n";
```

Copyright © 2011-2026 GB Research, LLC.

```
    } & _endl;

    // key name rule, can contain any characters, except '=' and spaces
    std::string key;
    auto key_rule = *_ws & +(_ - '=' - _endl - _hs) >> key;

    // value rules
    std::string value;

    // unquoted simple key value
    auto simple_value = *_hs & *(_ - _endl) >> value;

    // quoted value may have escaped quote chars
    auto quoted_value = *_hs & '"' & (*(_ - "\\\"" - '"') % +"\\\""_axe) >> value & '"';

    // key value can either be quoted or unquoted
    auto value_rule = quoted_value | simple_value;

    // rule for property line
    auto prop_line = key_rule & *_hs & '=' & value_rule & _endl >> [&]
    {
        std::cout << "    <property key=\"" << key << "\" value=\"" << value << "\"
/>\n";
    };

    // rule for comment
    auto comment = *_ws & ';' & *(_ - _endl) & _endl;

    // rule for INI file
    auto ini_file = *comment & *(section & *(prop_line | comment)) & *_ws & _z
        >> [&] { close_section(open_section); }
        | axe::r_fail([](auto i1, auto i2, auto i3)
    {
        std::cerr << "\nIni file contains errors, indicated by !\n";
        std::cerr << std::string(i1, i2) << "!" << std::string(i2, i3);
    });

    // do parsing
    ini_file(begin, end);
}
```

Now we can test this parser on a simple example:

```
void test_ini()
{
    std::string text{ R"**(
    ; This is a test of ini file
    [section1]
    key1 = 5
    key2 = value
    ; this is comment
    [section2]
    key = " quoted value: \"3\" "

)**" };
```

Copyright © 2011-2026 GB Research, LLC.

```
    ini(text.begin(), text.end());
}
```

It should print the following:

```
<section1>
    <property key="key1" value="5" />
    <property key="key2" value="value" />
</section1>
<section2>
    <property key="key" value=" quoted value: \"3\" " />
</section2>
```

---

7. INI file: Wikipedia

Command line parser

In this example we create a parser for command line and create semantic actions to extract keys and parameters.

This command line contains executable name, followed by optional key-value pairs, followed by optional parameters. In EBNF the grammar would look like this:

```
command_line ::= executable {"-" key [ = value]}* {parameter}*
```

The following function parses such command line and prints key-value pairs and parameters:

```
void parse_cmd_line(const std::string& cmd)
{
    using namespace axe;
    using namespace axe::shortcuts;

    // data to be gathered
    std::string exe_name;
    std::map<std::string, std::string> key_map;
    std::vector<std::string> parameters;

    // executable rule: file.extension
    auto executable = r_alnumstr() & *('.' & r_alnumstr());

    // key rule
    std::string key_name;
    auto key = r_alnumstr() >> key_name >> [&]() { key_map[key_name]; };

    // value rule
    auto value = r_alnumstr() >> [&](auto i1, auto i2)
    { key_map[key_name] = std::string(i1, i2); };

    // parameter rule
    auto parameter = r_alnumstr() >> e_push_back(parameters);
```

Copyright © 2011-2026 GB Research, LLC.

```
    // rule for key-value pairs
    auto key_value = '-' & key & ~(*_s & '=' & *_s & value);
    auto command_line = executable >> exe_name
        & *(+_s & key_value)
        & *(+_s & parameter) & _endl
        | r_fail([](auto, auto i2, auto i3)
    {
        std::cout << "Failed to parse portion of command line: " << std::string(i2, i3)
            << std::endl;
    });

    auto result = command_line(cmd.begin(), cmd.end());

    if (result.matched)
    {
        std::cout << "Matched string: " << std::string(cmd.begin(), result.position)
            << std::endl;
        std::cout << "Executable: " << exe_name << std::endl;
        std::cout << "Key-value pairs:" << std::endl;
        for (auto& i : key_map)
            std::cout << "\t" << i.first << " = " << i.second << std::endl;
        std::cout << "Parameters:" << std::endl;
        for (auto& p : parameters)
            std::cout << "\t" << p << std::endl;
    }
}

void test_cmd()
{
    std::cout << "----------------------------------------------------------test_cmd:\n";
    parse_cmd_line("command.exe -t=123 -h -p = p_value one two three");
    std::cout << "-------------------------------------------------------------------\n";
}
```

Windows Path Parser

This example shows how to create a parser for windows path format. Windows path starts with a letter followed by ':' or with "\\" followed by server name. Any characters are allowed, except "/?<>\\:*|\""[8] The path can be enclosed in double quotes, in which case it can contain spaces. The following function extracts and prints all paths found in a comma separated list.

```
template<class I>
void print_paths(I i1, I i2)
{
    using namespace axe;
    using namespace axe::shortcuts;
    // spaces are allowed in quoted paths only
    // illegal path characters, and the comma separating the paths
    auto illegal = r_any("/?<>\\:*|\",");
    // end of line characters
    auto endl = r_any("\n\r");
    // define path characters
    auto path_chars = _ - illegal - _hs - endl;
    // windows path can start with a server name or letter
```

Copyright © 2011-2026 GB Research, LLC.

```
    auto start_server = "\\\\" & +path_chars - '\\';
    auto start_drive = r_alpha() & ':';
    auto simple_path = (start_server | start_drive) & *('\\' & +path_chars);
    auto quoted_path = '"' & (start_server | start_drive) &
        *('\\' & +(_hs | path_chars)) & '"';
    // path can be either simple or quoted
    auto path = simple_path | quoted_path;
    // rule to extract all paths: the text between paths, separated by the paths
    std::vector<std::wstring> paths;
    auto extract_paths = *(_ - path) % (path >> e_push_back(paths)) & _z;
    // perform extraction
    extract_paths(i1, i2);
    // print extracted paths
    std::wcout << L"\nExtracted paths:\n";
    std::for_each(paths.begin(), paths.end(),
        [](const std::wstring& s)
    {
        std::wcout << s << L'\n';
    });
}

void test_paths()
{
    std::wcout << L"------------------------------------------------------------
test_paths:";
    std::string text(R"*(not-a-path, c:\a\b\c, \\server\a\b\c, "c:\quoted path")*");
    print_paths(text.begin(), text.end());
    std::wcout << L"------------------------------------------------------------------------
\n";
}
```

Output:

```
------------------------------------------------------------test_paths:
Extracted paths:
c:\a\b\c
\\server\a\b\c
"c:\quoted path"
------------------------------------------------------------------------
```

8. Naming Files, Paths, and Namespaces

Roman Numerals

Roman numerals stem from the numeral system of ancient Rome. They are based on certain letters of the alphabet which are combined to signify the sum (or, in some cases, the difference) of their values[9]. Parsing roman numerals can be done in different ways. This example shows a simple parser, which takes a string of roman numerals separated by spaces, converts and prints them.

```
void parse_roman(const std::string& txt)
{
    auto result = 0u;
```

Copyright © 2011-2026 GB Research, LLC.

```
    // thousands
    auto thousands = r_many("M"_axe >> [&result]{ result += 1000; }, 0, 3);
    // hundreds
    auto value100 = "C"_axe >> [&result] { result += 100; };
    auto value400 = "CD"_axe >> [&result] { result += 400; };
    auto value500 = "D"_axe >> [&result] { result += 500; };
    auto value900 = "CM"_axe >> [&result] { result += 900; };
    auto hundreds = value400 | value900 | ~value500 & r_many(value100, 0, 3);
    // tens
    auto value10 = "X"_axe >> [&result] { result += 10; };
    auto value40 = "XL"_axe >> [&result] { result += 40; };
    auto value50 = "L"_axe >> [&result] { result += 50; };
    auto value90 = "XC"_axe >> [&result] { result += 90; };
    auto tens = value90 | value40 | ~value50 & r_many(value10, 0, 3);
    // ones
    auto value1 = "I"_axe >> [&result] { result += 1; };
    auto value4 = "IV"_axe >> [&result] { result += 4; };
    auto value5 = "V"_axe >> [&result] { result += 5; };
    auto value9 = "IX"_axe >> [&result] { result += 9; };
    auto ones = value9 | value4 | ~value5 & r_many(value1, 0, 3);
    // a string of roman numerals separated by spaces
    auto roman = ((thousands & ~hundreds & ~tens & ~ones)
        >> [&result] { std::cout << result << ' '; result = 0; })
        % +_s;

    parse(roman, txt);
}

void test_roman()
{
    // test parser
    std::cout << "------------------------------------------------------------
test_roman:\n";
    std::string txt("I MMCCCLVI MMMCXXIII XXIII LVI MMMCDLVII DCCLXXXVI DCCCXCIX
MMMDLXVII");
    std::cout << txt << "\n";
    parse_roman(txt);
    std::cout << "\n------------------------------------------------------------------------
\n";
}
```

Output:

```
------------------------------------------------------------test_roman:
I MMCCCLVI MMMCXXIII XXIII LVI MMMCDLVII DCCLXXXVI DCCCXCIX MMMDLXVII
1 2356 3123 23 56 3457 786 899 3567
------------------------------------------------------------------------
```

9. Roman numerals: Wikipedia

Copyright © 2011-2026 GB Research, LLC.

JSON Parser

JSON is a lightweight text-based open standard designed for human-readable data interchange. It is derived from the JavaScript scripting language for representing simple data structures and associative arrays, called objects. Despite its relationship to JavaScript, it is language-independent, with parsers available for most languages.[10]

This example demonstrates how to use polymorphic parser rule `axe::r_rule` to define recursive rules.

```
template<class I>
void parse_json(I begin, I end)
{
    using namespace axe::shortcuts;

    auto json_hex = axe::r_many(axe::r_hex(), 4);
    auto json_escaped = "\""_axe | '\\' | '/' | 'b' | 'f'
        | 'n' | 'r' | 't' | 'u' & json_hex;
    auto json_char = _ - '"' - '\\' | '\\' & json_escaped;
    auto json_string = '"' & *json_char & '"';
    // definition of json_value requires recursion
    // neither 'auto' declaration, nor lambda functions allow recursion
    // instead one can create function object with recursion in operator()
    // or use polymorphic r_rule class, which performs type erasure
    axe::r_rule<I> json_value;

    // json_value must be wrapped with std::ref because rules are taken by value
    // an empty array is checked first, so no value is attempted for "[]"
    auto json_array = *_ws & '[' & *_ws
        & (']'_axe | (*_ws & std::ref(json_value) & *_ws) % ',' & ']');

    auto json_record = *_ws & json_string & *_ws
        & ':' & *_ws & std::ref(json_value) & *_ws;

    auto json_object = *_ws & '{' & ~(json_record % ',') & *_ws & '}';

    json_value = json_string | _double | json_object | json_array
        | "true" | "false" | "null";

    parse(json_object >> [](auto i1, auto i2)
    {
        std::cout << "JSON object parsed: " << std::string(i1, i2);
    } & _z
        | axe::r_fail([](auto i1, auto i2, auto i3)
    {
        std::cout << "parsing failed at place pointed by !\n"
            << std::string(i1, i2) << '!' << std::string(i2, i3);
    }), begin, end);
}

void test_json()
{
    std::cout << "------------------------------------------------------------test_json:\n";
    std::string str(R"*(
{
"category": 1,
```

Copyright © 2011-2026 GB Research, LLC.

```
"sub-category": 1.1,
"name": "inventory",
"tags": ["warehouse","inventory"],
"vehicles" :
[
        {
        "id": 123456789,
        "make": "Honda",
        "model": "Ridgeline",
        "trim": "RTL",
        "price": 32616,
        "tags": ["truck","V6","4WD"]
        },
        {
        "id": 748201836,
        "make": "Honda",
        "model": "Pilot",
        "trim": "Touring",
        "price": 38042,
        "tags": ["SUV","V6","4WD"]
        }
]
})*");

    parse_json(str.begin(), str.end());
    std::cout << "\n-------------------------------------------------------------------------
\n";
}
```

Running this program produces the following parsed text:

```
------------------------------------------------------------test_json:
JSON object parsed:
{
"category": 1,
"sub-category": 1.1,
"name": "inventory",
"tags": ["warehouse","inventory"],
"vehicles" :
[
        {
        "id": 123456789,
        "make": "Honda",
        "model": "Ridgeline",
        "trim": "RTL",
        "price": 32616,
        "tags": ["truck","V6","4WD"]
        },
        {
        "id": 748201836,
        "make": "Honda",
        "model": "Pilot",
        "trim": "Touring",
        "price": 38042,
        "tags": ["SUV","V6","4WD"]
        }
]
```

Copyright © 2011-2026 GB Research, LLC.

```
}
----------------------------------------------------------------------
```

This grammar is kept short. The JSON grammar in `test/jason_test.cpp` follows RFC 8259: unescaped string characters must be well-formed UTF-8 (`r_utf8`) and not control characters, UTF-16 escapes must pair a high surrogate with a low surrogate, any value is allowed at the top level, and nesting is bounded with `r_depth_limit`.

---

10. JSON: Wikipedia

Replacement Parser

This example demonstrates how to create a parser to replace the portions of a container matching specified rule with the content of another container. The function returns a tuple, consisting of a new container and number of occurrences of target rule in the source container.

```
template<class Container, class Rule>
auto replace(const Container& source, Rule r, const Container& rep)
{
    using namespace axe;
    using namespace axe::shortcuts;

    Container result;
    size_t counter = 0;

    // copy_rule matches all characters except r and copies them in result
    auto copy_rule = *(_ - r) >> [&](auto i1, auto i2)
    {
        result.insert(result.end(), i1, i2);
    };
    // subst_rule matches r and copies replacement in result
    auto subst_rule = r >> [&]
    {
        result.insert(result.end(), rep.begin(), rep.end());
        ++counter;
    };
    // replace_rule is the rule to process source
    auto replace_rule = *subst_rule & *(copy_rule & subst_rule) & _z;
    // perform parsing and replacement
    replace_rule(source.begin(), source.end());
    // return result and number of occurrences of r replaced
    return std::make_tuple(std::move(result), counter);
}
```

Now using this function, we can write a function to replace all occurrences of a target string in the source string with replacement string.

```
auto replace(const std::string& source,
    const std::string& target,
    const std::string& replacement)
{
```

Copyright © 2011-2026 GB Research, LLC.

```
    return replace(source, axe::r_str(target), replacement);
}

void test_replacement()
{
    std::cout << "------------------------------------------------------------
test_replacement:\n";
    auto source = R"*(
    This example shows how to replace all occurencies of abracadabra
    with something good. You should only see abracadabra in the output string.
    )*";

    std::cout << source;
    auto replacement = replace(source, "abracadabra", "something good");
    std::cout << "\nreplaced " << std::get<1>(replacement) << " occurences\n";
    std::cout << std::get<0>(replacement);
    std::cout << "\n------------------------------------------------------------------------
\n";
}
```

A word counting program

This example demonstrates how to create a parser for word counting program. An island parser for this program consists of three rules: parsing words, numbers, and all other symbols. The parsers for those are _w, +_d, and

```
auto other = +(_ - _n - _w - +_d); // rule matching everything else
```

To make this program more useful, we also collect the dictionaries matching these rules. To do that we use extractors with lambda functions, like this:

```
+_d >> // extractor to count numbers
        [this](auto i1, auto i2)
        {
            ++nnumber_;
            numbers_.insert(token_t(i1, i2));
        }
```

When rule is matched, extractor with specified lambda function is called inserting token into the dictionary and incrementing the number.

The full code for this program is shown below.

```
using token_t = std::vector<unsigned char>;

class wc
{
    // counts
    unsigned nline_{ 0 }, nword_{ 0 }, nnumber_{ 0 }, nother_{ 0 };
    // dictionaries
    std::set<token_t> words_;
```

Copyright © 2011-2026 GB Research, LLC.

```
    std::set<token_t> numbers_;
    std::set<token_t> other_;

    friend std::ostream& operator<< (std::ostream& os, const wc& w);

public:
    template<class I>
    wc& operator() (I begin, I end)
    {
        using namespace axe::shortcuts;

        auto other = +(_ - _n - _w - +_d); // rule matching everything else

        auto file = // rule for parsing input file
            *(
                other >> // extractor to count other symbols
                [this](auto i1, auto i2)
                {
                    if(i1 != i2)
                    {
                        ++nother_;
                        other_.insert(token_t(i1, i2));
                    }
                }
                | _n >> [this](auto...) { ++nline_; }
                | +_d >> // extractor to count numbers
                    [this](auto i1, auto i2)
                    {
                        ++nnumber_;
                        numbers_.insert(token_t(i1, i2));
                    }
                | _w >> // extractor to count words
                    [this](auto i1, auto i2)
                    {
                        ++nword_;
                        words_.insert(token_t(i1, i2));
                    }
            )
            & ~_ws
            & _z;

        auto p = axe::parse(file, begin, end); // do the actual parsing

        return *this;
    }
};

std::ostream& operator<< (std::ostream& os, const wc& w)
{
    // calculate dictionary size
    auto get_size = [](const std::set<token_t>& s)
    {
        return std::accumulate(s.begin(), s.end(), 0ull,
        [](auto v, const token_t& str)
        {
            return v + str.size();
        });
    };
```

Copyright © 2011-2026 GB Research, LLC.

```
    // report statistics
    os << "\nlines:\t " << w.nline_
        << "\nwords:\t " << w.nword_ << ", unique: " << w.words_.size()
        << ", dictionary size: " << get_size(w.words_)
        << "\nnumbers: " << w.nnumber_ << ", unique: " << w.numbers_.size()
        << ", dictionary size: " << get_size(w.numbers_)
        << "\nother:\t " << w.nother_ << ", unique: " << w.other_.size()
        << ", dictionary size: " << get_size(w.other_)
        << "\n";
#ifndef NDEBUG
    os << "Unique words:";
    for(auto&& w : w.words_)
        os << "\n\t" << std::string(w.begin(), w.end());
    os << "\nUnique numbers:";
    for(auto&& w : w.numbers_)
        os << "\n\t" << std::string(w.begin(), w.end());
#endif
    return os;
}

int main(int argc, char* argv[])
{
    if(argc == 2)
    {
        std::ifstream s(argv[1], std::ios_base::binary);
        if(s)
        {
            std::istreambuf_iterator<char> begin(s.rdbuf()), end;
            token_t vec(begin, end);

            std::cout << "Read " << vec.size() << " characters from " << argv[1];
            std::cout.flush();
            std::cout << wc()(vec.begin(), vec.end()) << "\n";
        }
        else
        {
            std::cerr << "ERROR: failed to open file: " << argv[1] << "\n";
        }
    }
    else
    {
        std::cerr << "\nusage: " << argv[0] << " <filename>\n";
    }

    return 0;
}
```

Wildcard parser

This example demonstrates how you can create a parser dynamically. The wildcard function takes a string, which contain wildcard characters (asterisks)[11] and creates an AXE parser corresponding to it, and then it returns the parser from the function. In order to create the parser dynamically we can use a polymorphic rule `axe::r_rule<I>` and while parsing the user input use extractors to build the final rule.

Here is one way it can be implemented:

Copyright © 2011-2026 GB Research, LLC.

```
auto wildcard(const std::string& target)
{
    using namespace axe::shortcuts;
    using I = decltype(target.begin());
    axe::r_rule<I> result;

    // match any number of chars except '*'
    auto match_string = +(_ - '*');

    // make a rule to match an extracted string
    auto make_str_rule = [](auto i1, auto i2)
    {
        return axe::r_str(std::string(i1, i2));
    };

    // match wildcard starting with '*'
    auto first_rule = match_string >> [&](auto i1, auto i2)
    {
        result = make_str_rule(i1, i2);
    };

    // match wilcard starting with '*' followed by a string
    auto second_rule = "*"_axe & match_string >> [&](auto i1, auto i2)
    {
        result = result & axe::r_find(make_str_rule(i1, i2));
    };

    // match wildcard ending with '*'
    auto third_rule = "*"_axe >> [&]
    {
        result = result & *_;
    };

    auto wc_rule = first_rule & *second_rule & ~third_rule
        | +second_rule & ~third_rule
        | third_rule;

    if (!parse(wc_rule & _z, target).matched)
        throw std::runtime_error("invalid wildcard: " + target);

    return result;
}
```

The above code parses wildcard target string and replaces normal characters with AXE `r_str` rules and asterisks with AXE `r_find` rules, which skip over the sequence of characters that do not match the substring.

Now we can write a test finding all substrings matching a given wildcard, similar to grep utility:

```
void test_wildcard()
{
    std::cout << "------------------------------------------------------------
test_wildcard:\n";
    auto grep = [](auto& wc, auto& text)
    {
        std::cout << "\n> grep \"" << wc << "\"\n";
```

Copyright © 2011-2026 GB Research, LLC.

```
        auto print = [](auto i1, auto i2)
        {
            std::cout << std::string(i1, i2) << "\n";
        };

        // find all substrings matching the wildcards
        auto res = axe::parse(+axe::r_find(wildcard(wc) >> print), text);
        if (!res.matched)
            std::cout << wc << " -- not found\n";
    };

    const std::string text(R"**(The asterisk in a wildcard matches any character zero or
more times.
For example, "comp*" matches anything beginning with "comp" which means
"comp", "complete", and "computer" are all matched.)**");

    std::cout << text << "\n";

    grep("asterisk*wildcard", text);
    grep("*asterisk*times", text);
    grep("\"comp*\"", text);
    grep("abracadabra", text);
    std::cout << "\n-------------------------------------------------------------------
\n";
}
```

The `test_wildcard` function will print the following results:

```
------------------------------------------------------------test_wildcard:
The asterisk in a wildcard matches any character zero or more times.
For example, "comp*" matches anything beginning with "comp" which means
"comp", "complete", and "computer" are all matched.

> grep "asterisk*wildcard"
asterisk in a wildcard

> grep "*asterisk*times"
The asterisk in a wildcard matches any character zero or more times

> grep ""comp*""
"comp*"
"comp"
"comp"
"complete"
"computer"

> grep "abracadabra"
abracadabra -- not found

-------------------------------------------------------------------
```

11. Wikipedia: Wildcard character

Copyright © 2011-2026 GB Research, LLC.

Sources

The sources of AXE, with documentation comments in the headers, and the tests, which include the examples above, are at https://github.com/gbresearch/axe. `axe.h` includes all the other headers.

Copyright © 2011-2026 GB Research, LLC.